

Calculando Hashes

1. Visão Geral

Esta atividade demonstra o uso prático de funções de hash unidirecionais para garantir a integridade de dados. Nele, explora-se a geração de assinaturas digitais por meio de utilitários de linha de comando (openssl e sha256sum/sha512sum), observando na prática o efeito avalanche (*avalanche effect*) e a validação do download correto de arquivos de imagem.

2. Objetivo

- Gerar hashes de arquivos de texto utilizando o OpenSSL com algoritmos da família SHA-2 (SHA-256 e SHA-512).
- Observar a sensibilidade das funções de hash a mínimas alterações no arquivo de entrada (Efeito Avalanche).
- Comparar os resultados de geração de hashes entre diferentes utilitários de CLI (openssl vs shaXsum).
- Validar a integridade de um arquivo baixado (sample.img) comparando o hash calculated com a assinatura original fornecida (.sig).

3. Ambiente

- Sistema Operacional: Linux (Máquina Virtual CyberOps Workstation)
- Plataforma: Cisco Networking Academy (CyberOps)
- Topologia: Estação de trabalho isolada (Localhost)
- Tecnologias: Hashing unidirecional, Família SHA-2 (SHA-256, SHA-512)
- Ferramentas: Terminal Bash, OpenSSL, sha256sum, sha512sum, Editor nano

4. Cenário

Uma analista de segurança precisa garantir que arquivos transmitidos ou armazenados não foram alterados por erros de transmissão ou manipulação maliciosa. O cenário exige a validação da integridade de uma carta em formato texto

(letter_to_grandma.txt) e a verificação do download íntegro de uma imagem de sistema (sample.img).

5. Execução

5.1 Etapa 1 — Geração de Hash Original com OpenSSL (SHA-256)

Navegação até o diretório dos arquivos do laboratório, leitura do arquivo original e geração do hash SHA-256 via OpenSSL:

```
cd /home/analyst/lab.support.files/  
cat letter_to_grandma.txt  
openssl sha256 letter_to_grandma.txt
```

- **Saída Obtida:**

```
SHA256(letter_to_grandma.txt) =  
deff9c9bbece44866796ff6cf21f2612fbb77aa1b2515a900bafb29be118080b  
...
```

5.2 Etapa 2 — Modificação do Arquivo e Avaliação do Efeito

Avalanche

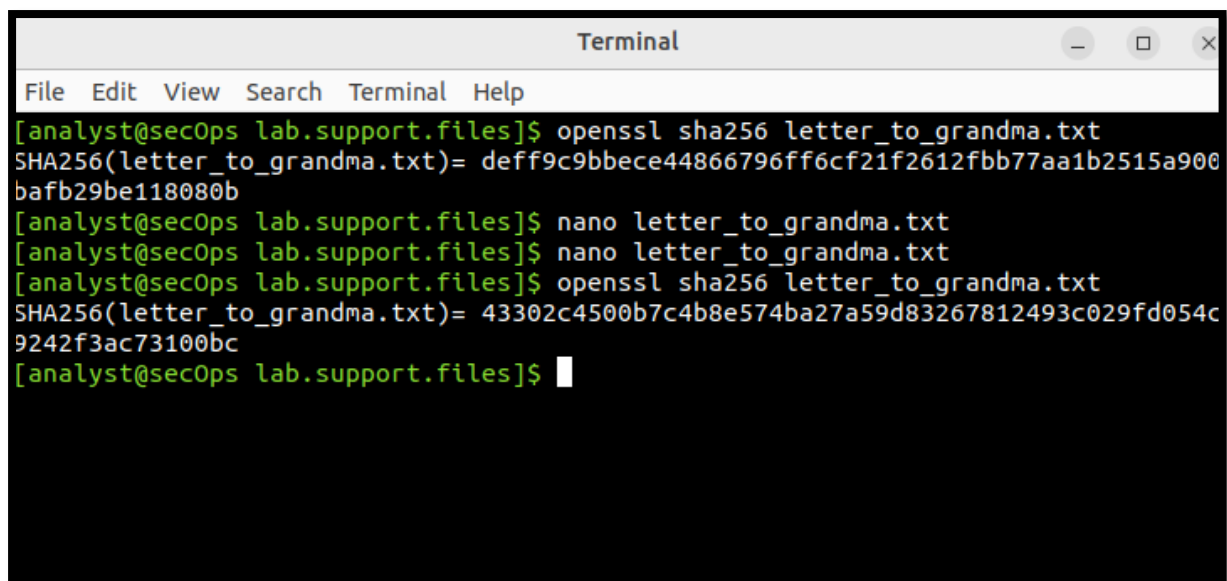
Abertura do arquivo com o editor nano para alterar um único caractere da mensagem ('m' de "Grandma" para 'p', resultando em "Grandpa"). Em seguida, o hash SHA-256 foi recalculado:

```
nano letter_to_grandma.txt
```

Alteração	realizada:	"Hi Grandma"	->	"Hi Grandpa"
openssl	sha256			letter_to_grandma.txt

- **Nova Saída Gerada:**

```
SHA256(letter_to_grandma.txt) =  
43302c4500b7c4b8e574ba27a59d83267812493c029fd054c9242f3ac73100bc  
...
```

A terminal window titled "Terminal" with a menu bar (File, Edit, View, Search, Terminal, Help). The prompt is [analyst@secOps lab.support.files]\$. The first command is openssl sha256 letter_to_grandma.txt, resulting in SHA256(letter_to_grandma.txt)= deff9c9bbece44866796ff6cf21f2612fbb77aa1b2515a900bafb29be118080b. The second command is nano letter_to_grandma.txt, which is entered twice. The third command is openssl sha256 letter_to_grandma.txt, resulting in SHA256(letter_to_grandma.txt)= 43302c4500b7c4b8e574ba27a59d83267812493c029fd054c9242f3ac73100bc. The prompt is [analyst@secOps lab.support.files]\$.

```
Terminal
File Edit View Search Terminal Help
[analyst@secOps lab.support.files]$ openssl sha256 letter_to_grandma.txt
SHA256(letter_to_grandma.txt)= deff9c9bbece44866796ff6cf21f2612fbb77aa1b2515a900bafb29be118080b
[analyst@secOps lab.support.files]$ nano letter_to_grandma.txt
[analyst@secOps lab.support.files]$ nano letter_to_grandma.txt
[analyst@secOps lab.support.files]$ openssl sha256 letter_to_grandma.txt
SHA256(letter_to_grandma.txt)= 43302c4500b7c4b8e574ba27a59d83267812493c029fd054c9242f3ac73100bc
[analyst@secOps lab.support.files]$
```

Figura 1: Demonstração do Efeito Avalanche — alteração do caractere 'm' para 'p' e mudança completa no digest SHA-256.

5.3 Etapa 3 — Comparação com Ferramentas Nativas e SHA-512

Geração dos hashes utilizando um algoritmo de maior comprimento de bits (SHA-512) e comparação do OpenSSL com os binários dedicados do Linux (sha256sum e sha512sum):

```
openssl sha512 letter_to_grandma.txt
sha256sum letter_to_grandma.txt
sha512sum letter_to_grandma.txt
```

- **Saída openssl sha512:**

```
7c35db79a06aa30ae0f6de33f2322fd419560ee9af9cedeb6e251f2f1c4e99e0bbe5d2fc32ce501468891150e3be7e288e3e568450812980c9f8e3a31d3
```

- **Saída sha256sum:**

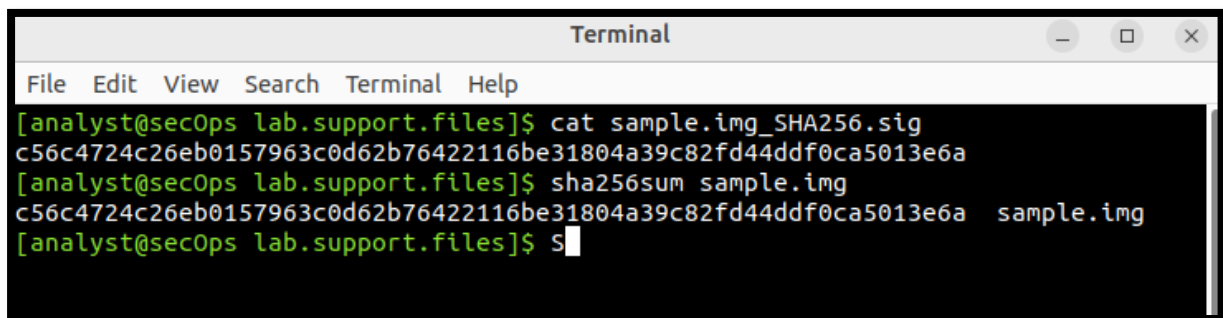
```
43302c4500b7c4b8e574ba27a59d83267812493c029fd054c9242f3ac73100bc
letter_to_grandma.txt
```

5.4 Etapa 4 — Verificação de Integridade de Arquivo Baixado (sample.img)

Comparação do valor contido no arquivo de assinatura original (sample.img_SHA256.sig) com o hash SHA-256 gerado diretamente do arquivo baixado:

```
cat sample.img_SHA256.sig
sha256sum sample.img
```

- **Hash da Assinatura (.sig):**
c56c4724c26eb0157963c0d62b76422116be31804a39c82fd44ddf0ca5013e6a
- **Hash Calculado do Arquivo (sample.img):**
c56c4724c26eb0157963c0d62b76422116be31804a39c82fd44ddf0ca5013e6a



```
Terminal
File Edit View Search Terminal Help
[analyst@secOps lab.support.files]$ cat sample.img_SHA256.sig
c56c4724c26eb0157963c0d62b76422116be31804a39c82fd44ddf0ca5013e6a
[analyst@secOps lab.support.files]$ sha256sum sample.img
c56c4724c26eb0157963c0d62b76422116be31804a39c82fd44ddf0ca5013e6a sample.img
[analyst@secOps lab.support.files]$ S
```

Figura 2: Validação de integridade do arquivo `sample.img`

6. Resultados

- **Detecção de Alteração:** A mudança de apenas um único caractere no arquivo de entrada alterou completamente o hash resultante (Efeito Avalanche).
- **Consistência de Ferramentas:** Os hashes calculados via openssl e utilitários nativos shaXsum resultaram exatamente nos mesmos valores hexadecimais, diferindo apenas no formato da saída na CLI.
- **Confirmação de Integridade:** O arquivo sample.img foi baixado com 100% de integridade, pois o hash calculado bateu exatamente com o hash disponibilizado pela fonte.

7. Análise Técnica

- **Propriedade determinística e Unidirecional:** O mesmo arquivo sempre gera a mesma cadeia de saída fixa (ex: 256 bits / 64 caracteres hexadecimais para SHA-256). No entanto, é impossível reverter o hash para obter o texto original.
- **Efeito Avalanche (*Avalanche Effect*):** Na matemática dos algoritmos criptográficos como o SHA-2, a menor alteração em 1 bit da entrada afeta substancialmente a propagação de dados em suas rodadas internas de processamento, produzindo um resultado totalmente diferente.
- **Diferenças de Interface vs Algoritmo:** openssl e sha256sum são apenas implementações de software/front-ends diferentes para o mesmo cálculo matemático padronizado. Por isso, produzem o mesmo digest final.

8. Conclusão

O laboratório demonstrou a relevância prática dos algoritmos de hash para a auditoria de integridade da informação em sistemas operacionais e ambientes de redes. Foi comprovado que a validação via hash identifica com precisão modificações acidentais ou maliciosas em arquivos de qualquer formato.

9. Competências Desenvolvidas

- Verificação de Integridade de Dados
- Operação de utilitários criptográficos em ambiente Linux (openssl, sha256sum, sha512sum)
- Identificação do Efeito Avalanche em algoritmos de Hash
- Validação de integridade de software/arquivos em auditoria de segurança

10. Evidências

- **Hash SHA-256 Original (letter_to_grandma.txt):**
deff9c9bbece44866796ff6cf21f2612fbb77aa1b2515a900bafb29be118080b
- **Hash SHA-256 Pós-Modificação (letter_to_grandma.txt):**
43302c4500b7c4b8e574ba27a59d83267812493c029fd054c9242f3ac73100bc
 - **Hash SHA-256 de Validação (sample.img):**
c56c4724c26eb0157963c0d62b76422116be31804a39c82fd44ddf0ca5013e6a

11. Referências

Cisco Networking Academy — CCNA CyberOps / Cybersecurity Operations

NIST FIPS PUB 180-4 — Secure Hash Standard (SHS / SHA-2)

Documentação do Projeto OpenSSL (man openssl)